Influence of MPI Domain Decomposition on a Multi-GPU Shallow-Water Solver

High Performance Computing for Weather and Climate – Work Project

Ramura Gassler Dennis Eberli Timon Maeder

ETH Zurich, Switzerland

August 28, 2026

Abstract

This project investigates how the shape of a two-dimensional MPI domain decomposition affects the performance of a multi-GPU shallow-water-equation solver. The starting point is an existing Julia code based on `ParallelStencil.jl` and `ImplicitGlobalGrid.jl` (IGG), which executes the same stencil kernels on CPUs and GPUs and uses IGG for distributed-memory parallelization. Using a fixed global problem and a fixed number of GPUs, we compare decompositions ranging from strongly elongated process grids to more balanced layouts. The report focuses on the relationship between local subdomain geometry, boundary-to-volume ratio, communication cost, and achieved cell-update rate.

1 Introduction and Objectives

Many explicit finite-volume solvers spend most of their runtime updating local grid cells using data from a small stencil. This structure maps efficiently to GPUs, but simulations that exceed the memory or computational capacity of one accelerator require a distributed-memory decomposition. The global grid is divided into subdomains, each subdomain is assigned to one MPI rank and typically one GPU, and neighboring ranks exchange halo cells after every update that changes stencil inputs.

The topology of the process grid determines the shape of each local subdomain. Although two topologies may use the same number of MPI ranks and perform almost the same arithmetic work, they can expose different interface lengths and therefore different communication volumes. This project studies that effect using a two-dimensional shallow-water-equation (SWE) solver adapted from the multi-XPU implementation in the `ShallowWater4PDEonGPU` repository [1].

The objectives are:

1. understand which distributed-memory tasks are handled implicitly by IGG;
2. reproduce the required functionality directly with `MPI.jl`;
3. simplify the physical experiment so that the benchmark isolates domain-decomposition effects;
4. make the solver modular with respect to global grid size, process topology, number of time steps, warm-up iterations, and repeated runs;
5. benchmark several 16-rank GPU topologies on Santis; and
6. relate the measured runtime to local subdomain shape and a simple communication-volume model.

The primary research question is:

> *For a fixed global grid and a fixed number of GPUs, how does the MPI process topology influence the runtime and throughput of the shallow-water solver?*

A secondary question is whether the manually implemented MPI layer reaches performance comparable to the original IGG-based implementation while retaining transparent control over decomposition and communication.

2 Background

2.1 Shallow-water equations

The shallow-water equations (SWE) describe depth-averaged free-surface flows for which the horizontal length scales are much larger than the characteristic water depth. They are relevant to weather and climate modelling because they capture important geophysical processes, such as horizontal transport and gravity-wave propagation, at substantially lower computational cost than the full three-dimensional equations.

The two-dimensional balance law is

$$\frac{\partial \boldsymbol{U}}{\partial t} + \frac{\partial \boldsymbol{F}(\boldsymbol{U})}{\partial x} + \frac{\partial \boldsymbol{G}(\boldsymbol{U})}{\partial y} = \boldsymbol{S}(\boldsymbol{U}, z), \tag{1}$$

with fluxes

$$\boldsymbol{F}(\boldsymbol{U}) = \begin{bmatrix} hu \\ hu^2/h + \frac{1}{2}gh^2 \\ huv/h \end{bmatrix}, \qquad \boldsymbol{G}(\boldsymbol{U}) = \begin{bmatrix} hv \\ huv/h \\ hv^2/h + \frac{1}{2}gh^2 \end{bmatrix}, \tag{2}$$

and the bathymetry source term

$$\boldsymbol{S}(\boldsymbol{U}, z) = \begin{bmatrix} 0 \\ -gh\,\dfrac{\partial z}{\partial x} \\ -gh\,\dfrac{\partial z}{\partial y} \end{bmatrix}. \tag{3}$$

The source term represents the acceleration caused by gradients in the bottom topography $z(x, y)$. In this project, the bathymetry is set to $z = 0$, such that

$$\frac{\partial z}{\partial x} = \frac{\partial z}{\partial y} = 0 \qquad \Longrightarrow \qquad \boldsymbol{S} = \boldsymbol{0}. \tag{4}$$

The detailed numerical discretization, including the conservative well-balanced Rusanov solver, wet/dry treatment, and timestep restrictions, can be found in the original SWE repository [1]. Since this project focuses on distributed-memory parallelization, these numerical components are not repeated here.

2.2 ParallelStencil and ImplicitGlobalGrid

The solver uses `ParallelStencil.jl` to define architecture-independent stencil kernels. The same numerical kernels can run on multi-threaded CPUs or CUDA GPUs, depending on the selected backend.

The original multi-XPU implementation uses `ImplicitGlobalGrid.jl` (IGG) for distributed-memory parallelization. IGG provides the Cartesian process topology, local-to-global index mapping, neighboring-rank communication, halo updates, global reductions, and distributed gathers.

In this project, the numerical kernels are kept largely unchanged, while the required IGG functionality is replaced by an explicit implementation using `MPI.jl`.

2.3 MPI domain decomposition

In domain decomposition, the global grid is divided into smaller rectangular subdomains. Each subdomain is assigned to one MPI rank and, for the GPU benchmarks, to one GPU.

For a global interior grid of size $N_x \times N_y$ and a process topology $P_x \times P_y$, the local interior size is

$$n_x = \frac{N_x}{P_x}, \qquad n_y = \frac{N_y}{P_y}. \tag{5}$$

Each local array contains an additional halo layer that stores values received from neighboring ranks. For a one-cell halo, the amount of communicated boundary data is approximately proportional to

$$V_{\text{halo}} \propto 2n_x + 2n_y, \tag{6}$$

whereas the computational work scales with the local area $n_x n_y$. A simple communication-to-computation indicator is therefore

$$\chi = \frac{2(n_x + n_y)}{n_x n_y}. \tag{7}$$

For a fixed local area, square-like subdomains minimize this ratio. For a rectangular global domain, the preferred MPI topology is expected to approximately follow the global aspect ratio.

3 Methodology

3.1 Starting point and simplifications

The numerical code is adapted from `src/xpu/2d_swe_multi_xpu_wb.jl` in the existing SWE repository [1]. The original code supports well-balanced flow over non-flat bathymetry, wet/dry interfaces, large-domain visualization, and a multi-XPU implementation based on IGG. The project version retains the main finite-volume kernels but applies the following simplifications:

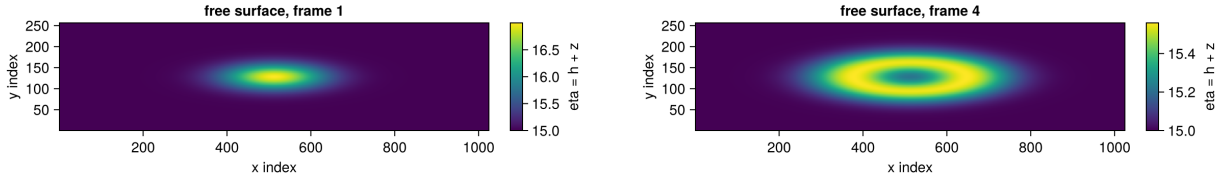
- flat bathymetry, $z(x, y) = 0$;
- no topography-file loading or preprocessing;
- one smooth Gaussian perturbation as the initial condition;
- no in-solver plotting during benchmarks;
- timing restricted to the repeated simulation loop; and
- blocking halo exchange without hidden communication.

The initial free surface is

$$\eta_0(x, y) = h_0 + A \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right), \tag{8}$$

with the current code using $h_0 = 15$, $A = 2$, and $\sigma = \max(L_x, L_y)/12$ on a domain of size $L_x = L_y = 50$. Initial momenta are zero.

This setup produces a radially propagating wave while keeping initialization identical for every topology. It therefore provides nontrivial stencil work without introducing topology-dependent input data.



(a) Initial state with Gaussian perturbation.

(b) Propagating wave after several timesteps.

Figure 1: Visualisation of the shallow-water simulation.

3.2 Manual Cartesian communicator and local domains

The manual solver requires a user-defined topology (P_x, P_y). The global interior dimensions must be divisible by the respective process-grid dimensions. Local sizes including one halo cell on each side are

$$n_x^{\text{alloc}} = \frac{N_x}{P_x} + 2, \qquad n_y^{\text{alloc}} = \frac{N_y}{P_y} + 2. \tag{9}$$

The Cartesian communicator and direct neighbors are created explicitly:

```
dims_mpi = collect(topology)
comm_cart = MPI.Cart_create(
    MPI.COMM_WORLD,
    dims_mpi;
    periodic=(false, false),
    reorder=false,
)

me = MPI.Comm_rank(comm_cart)
```

```
dims, periods, coords = MPI.Cart_get(comm_cart)
neighbors_x = MPI.Cart_shift(comm_cart, 0, 1)
neighbors_y = MPI.Cart_shift(comm_cart, 1, 1)

is_left   = neighbors_x[1] == MPI.PROC_NULL
is_right  = neighbors_x[2] == MPI.PROC_NULL
is_bottom = neighbors_y[1] == MPI.PROC_NULL
is_top    = neighbors_y[2] == MPI.PROC_NULL
```

Listing 1: Explicit MPI Cartesian topology used in the manual solver.

Using `MPI.PROC_NULL` distinguishes inter-rank interfaces from physical domain boundaries. Boundary-condition kernels are launched only by ranks that lie on the corresponding outer edge.

3.3 Manual global-index mapping

IGG's global-coordinate helpers are replaced by a direct mapping based on Cartesian rank coordinates. The ranges include halos, so adjacent rank ranges overlap exactly where halo values are stored.

```
@views function get_global_indices(nx, ny, coords)
    nx_local = nx - 2
    ny_local = ny - 2

    ix_start = coords[1] * nx_local + 1
    iy_start = coords[2] * ny_local + 1

    ix_end = ix_start + nx - 1
    iy_end = iy_start + ny - 1

    return (ix_start:ix_end, iy_start:iy_end)
end
```

Listing 2: Manual local-to-global index mapping.

The global indices are converted to physical coordinates and used to evaluate the Gaussian initial condition independently on every rank. This avoids a global initialization followed by scatter communication.

3.4 Blocking halo exchange

Reusable send and receive buffers are allocated once per field. This avoids repeated allocation inside the time-stepping loop and works for both CPU arrays and CUDA arrays through `similar`. For each field, the exchange proceeds in two stages:

1. pack and exchange the left/right interior boundaries;
2. unpack the received x halos;
3. pack and exchange the bottom/top boundaries, now including updated corner information; and
4. unpack the received y halos.

```
@views begin
    buffers.send_left  .= A[2, :]
    buffers.send_right .= A[end-1, :]
end
backend_synchronize()

MPI.Sendrecv!(buffers.send_left, left, tag_base,
              buffers.recv_right, right, tag_base, comm_cart)
MPI.Sendrecv!(buffers.send_right, right, tag_base + 1,
```

```
            buffers.recv_left, left, tag_base + 1, comm_cart)

@views begin
    if right != MPI.PROC_NULL
        A[end, :] .= buffers.recv_right
    end
    if left != MPI.PROC_NULL
        A[1, :] .= buffers.recv_left
    end
end
```

Listing 3: Core structure of the blocking halo exchange.

The same function is called for h, hu, hv, and the draining-timestep field, using separate MPI tag ranges. On the GPU backend, explicit synchronization ensures that packing kernels complete before MPI reads a send buffer and that unpacking completes before later kernels consume the halos.

The implementation uses blocking `MPI.Sendrecv!`. It therefore does not overlap halo communication with interior computation. This is a deliberate baseline choice: differences between process topologies can be interpreted without an additional overlap strategy, while communication hiding remains a possible later optimization.

3.5 Manual gather

For output and decomposition visualization, each rank flattens its local interior and participates in `MPI.Gatherv!`. Rank 0 then places the chunks according to Cartesian coordinates:

```
sendbuf = vec(local_interior)
counts = fill(length(sendbuf), nprocs)

if me == 0
    recvbuf_data = similar(sendbuf, sum(counts))
    recvbuf = MPI.VBuffer(recvbuf_data, counts)
    MPI.Gatherv!(sendbuf, recvbuf, comm_cart; root=0)

    offset = 1
    for rank in 0:(nprocs - 1)
        coords = MPI.Cart_coords(comm_cart, rank)
        ix0 = coords[1] * local_nx + 1
        iy0 = coords[2] * local_ny + 1
        chunk = @view recvbuf_data[offset:offset+counts[rank+1]-1]
        global_array[ix0:ix0+local_nx-1,
                     iy0:iy0+local_ny-1] .=
            reshape(chunk, local_nx, local_ny)
        offset += counts[rank+1]
    end
else
    MPI.Gatherv!(sendbuf, nothing, comm_cart; root=0)
end
```

Listing 4: Simplified manual gather and rank-ordered unpacking.

The current project restricts benchmarks to uniform local block sizes. Under this restriction the counts are equal, although `Gatherv` leaves room for later support of nonuniform decompositions. A general nonuniform implementation would additionally require rank-specific local dimensions and displacements.

3.6 Benchmark-oriented modularization

The solver provides three benchmark variants, each implemented as a separate Julia source file that shares the same numerical kernels but differs in how timing is collected.

1. **Walltime variant** (`manual.jl`): measures the total runtime of the complete time-stepping loop per run, repeated multiple times for each topology.
2. **Kernel-timing variant** (`manual_kernel_timing.jl`): inserts fine-grained timers around three kernel phases within each iteration, recording per-iteration kernel wall times. The halo exchange is excluded from these timers.
3. **Halo-timing variant** (`manual_halo_timing.jl`): records per-iteration wall time of the halo-exchange calls separately, allowing a direct comparison of communication cost between topologies.

All three variants accept command-line parameters for the most important run-time choices:

- `-nx`, `-ny`: global interior grid dimensions (including halo cells);
- `-topology` (or `-topo`): process topology (P_x, P_y);
- `-nt`: number of time steps;
- `-warmup`: number of untimed warm-up iterations before the measured loop;
- `-runs`: number of repeated runs (walltime variant only);
- `-benchmark`: enables timed execution and CSV output;
- `-benchdir`: CSV output path.

Each variant writes its results to a separate CSV file with a header that records solver type, topology, grid dimensions, and measured timings. The walltime variant additionally computes steps per second and cell updates per second (Equation 10).

The benchmark sequence is the same in all variants:

1. initialize MPI and construct the Cartesian communicator;
2. allocate all arrays and halo buffers once, before any timing;
3. evaluate the Gaussian initial condition independently on every rank, avoiding a global initialization followed by scatter;
4. execute untimed warm-up steps to reach a thermally settled state;
5. reset all fields to the initial state;
6. synchronize the device and execute an MPI barrier;
7. time only the main time-stepping loop;
8. use the maximum elapsed time over all ranks as the run time of that repetition; and
9. append one CSV row per repetition (walltime variant) or one row per iteration (kernel and halo variants).

Taking the maximum rank time rather than the mean is essential because the distributed simulation progresses at the speed of the slowest rank. A mean would hide load imbalance behind faster ranks that finish their communication earlier.

The reported throughput is

$$R_{\text{cell}} = \frac{N_x N_y N_t}{t_{\text{wall}}} \quad [\text{cell updates per second}] \,. \tag{10}$$

For strict interpretation, $N_x N_y$ should denote the global *interior* cell count. The current code uses the dimensions including the two outer halo/boundary cells. For the large benchmark grid the numerical difference is negligible.

Repeated runs share the same allocated arrays and are separated by a field reset and a device synchronization, so that memory-allocation overhead and kernel-launch transients are excluded from every measurement.

3.7 Validation strategy

Before the runtime benchmarks, the manual MPI implementation is validated against the original IGG-based solver to confirm correctness. Both solvers are run with identical global parameters and their initial water-depth field h is compared using the relative L2 error

$$\varepsilon = \frac{\|h_{\text{manual}} - h_{\text{IGG}}\|_2}{\|h_{\text{IGG}}\|_2}.$$

The test yields $\varepsilon = 0$ to machine precision, confirming that the manual Cartesian communicator, local-to-global index mapping, and gather routine produce the same initial state as the IGG reference. Because the stencil kernels are identical in both solvers (only the communication layer differs), the same agreement holds for all subsequent time steps. This validation addresses the prerequisite for both research questions: without matching fields the topology benchmarks (Question 1) would measure different numerics rather than different communication patterns, and the performance comparison with IGG (Question 2) would compare non-equivalent implementations.

3.8 Benchmark setup

Benchmarks were run on the Santis cluster with 4 and 16 MPI ranks, on one and four GPU nodes respectively. The global grid has a 4:1 aspect ratio. A decomposition is written as $P_x \times P_y$, where P_x is the number of ranks along the x direction and P_y along the y direction. For 16 ranks, the 4:1 grid yields five distinct decompositions.

Three types of timing were recorded for every topology:

- total walltime: the full iteration loop, 15 runs per topology;
- kernel timing: all kernel solvers, timed for each iteration of the main loop (200 iterations);
- halo timing: the halo exchange between ranks, timed for each iteration of the main loop (200 iterations).

4 Results

Results are reported for the benchmark setup described in the Methodology. The analysis focuses on the 16-rank case, for which the decomposition shapes leave more room for discussion.

4.1 Topology benchmark

Table 1 summarises the five process topologies and the resulting subdomain shapes for the 4:1 global grid. A visualisation of the decomposition layouts is provided in Appendix B.

The strip topologies (16×1, 1×16) expose each subdomain to at most two MPI neighbors, while the rectangular topologies (8×2, 2×8, 4×4) require communication in both coordinate directions and therefore contact up to three or four neighbors. The corresponding communication-to-computation indicators χ differ by a factor of 4 between the lowest value (0.98×10^{-3} for the 8x2 topology) and the highest value (3.97×10^{-3} for the 1x16). This would indicate a lower communication cost for the topology with square subdomains.

Table 1: Process topologies and their subdomain properties for the 16-rank benchmarks.

Topology($P_x \times P_y$)	Subdomain $n_x \times n_y$	Max. neighbors	$\chi = \frac{2(n_x+n_y)}{n_x n_y}$
16×1	2050 × 8194	2 (x only)	1.22×10^{-3}
8×2	4098 × 4098	3 (x and y)	0.98×10^{-3}
4×4	8194 × 2050	4 (x and y)	1.22×10^{-3}
2×8	16386 × 1026	3 (x and y)	2.07×10^{-3}
1×16	32770 × 514	2 (y only)	3.97×10^{-3}

Figure 2 shows the measured total effective throughput for the five topologies.

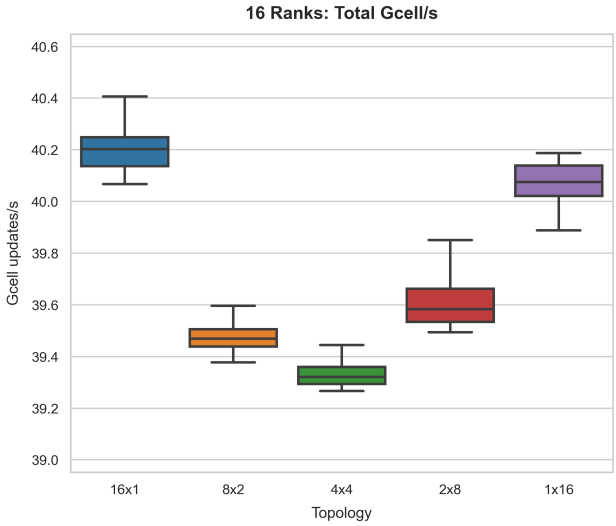


Figure 2: Total effective throughput per topology for the 16-rank benchmarks.

The absolute throughput differences are small: the spread between the fastest and slowest topology is approximately 0.09×10^{10} cell updates per second, which corresponds to roughly two percent of the mean throughput. Nevertheless, a clear ranking emerges: the strip topologies (16×1, 1×16) achieve the highest median throughput at approximately 4.02×10^{10} and 4.01×10^{10} cell updates per second, respectively, while the 4×4 decomposition trails at 3.93×10^{10}. The 2×8 and 8×2 topologies lie in between with 3.96×10^{10} and 3.95×10^{10}.

Figure 3 splits the per-iteration time into kernel computation and halo communication, and shows the communication share for each topology.

The kernel computation time varies only marginally between the topologies, confirming that the stencil work mainly depends on the global grid size and not the subdomain shape. Minimally higher kernel times per iteration are measured for the strip topologies (1×16 and 16×1). The difference is in the order of 0.1 ms per iteration. The halo communication time, however, varies more significantly: the strip topologies spend about 0.55 to 0.57 ms per iteration on halo exchange, while the rectangular topologies spend 0.70 to 0.75 ms. The communication share consequently ranges from about 8.3 percent (16×1) to 11.1 percent (8×2). We see comparable communication terms for both of the strip topologies as well as for all of the rectangular topologies.

4.2 Manual MPI versus IGG baseline

The overall performance of the manual MPI layer is compared with the original IGG-based implementation for several grid sizes. Figure 4 shows the throughput as the global problem size grows.

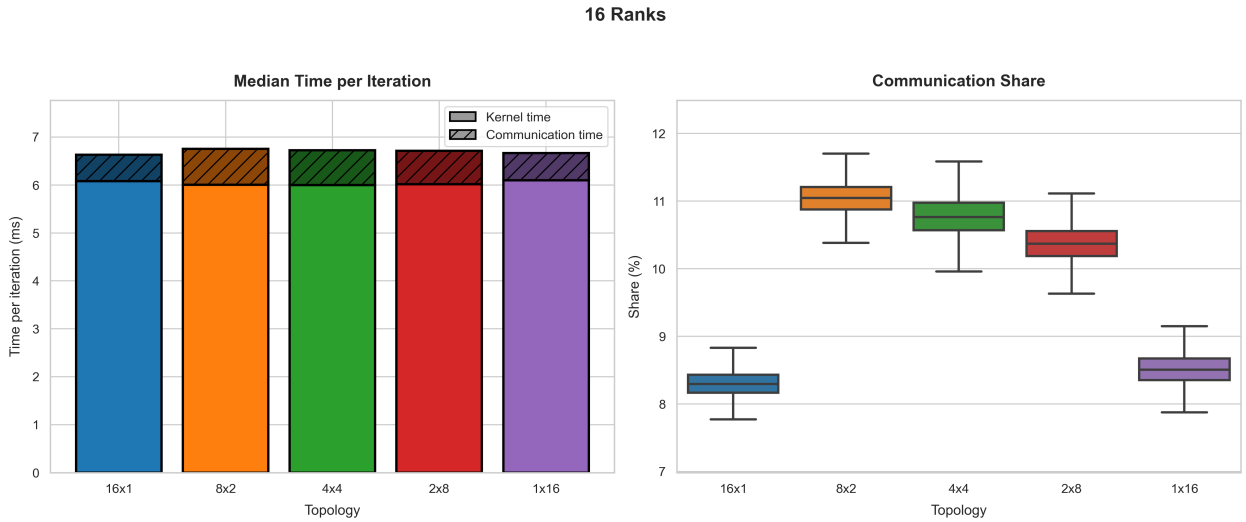


Figure 3: Median time per iteration split into kernel and halo communication, and communication share, for the 16-rank topologies.

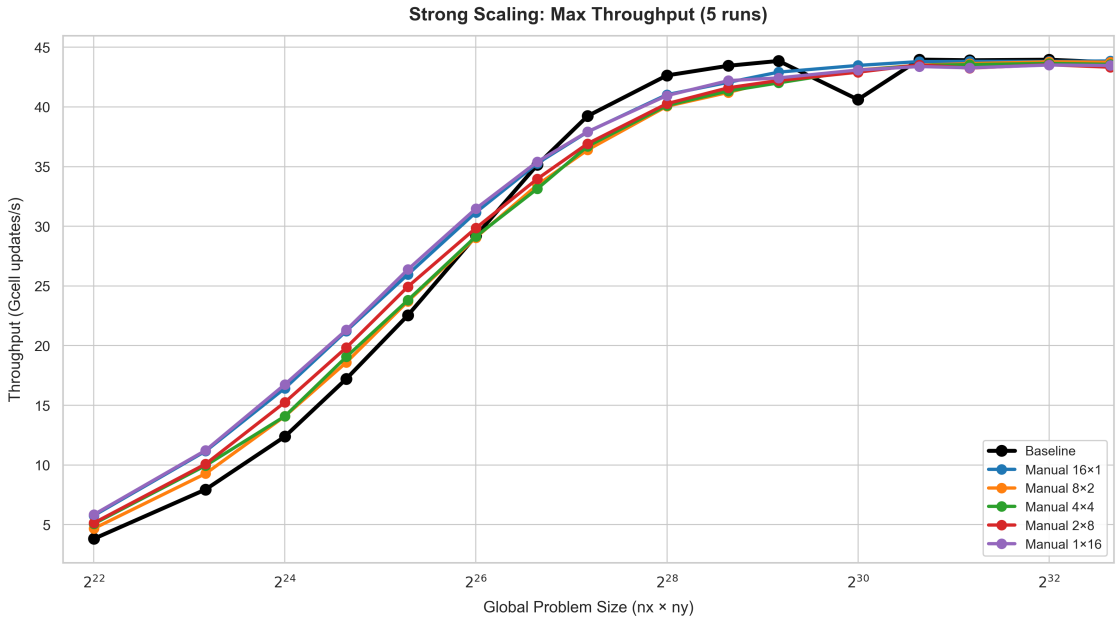


Figure 4: Throughput for the IGG baseline and the manual topologies as a function of the global problem size.

At small problem sizes (below 10^8 or 2^{26} cells) the manual implementation outperforms the baseline by a factor of 1.1 to 1.4. The advantage shrinks as the grid grows and reverses between 10^8 and 10^9 cells, where the baseline achieves slightly higher throughput (ratio 0.94 to 0.97). For the largest grids, both implementations converge to nearly identical performance at 44 GCell updates/s. The performance plot shows the maximum throughput of 5 runs per grid size. More samples would help quantify the variability between runs.

5 Discussion

5.1 Effect of subdomain shape

It is shown in Figure 2 that the sub-domain shape has nearly negligible effects on the time per iteration for the computation of the stencil update. Any difference in per-iteration time can be attributed almost entirely to the halo update.

The simple χ indicator introduced in chapter 2.3 assumes the communication cost scales with the perimeter-to-area ratio of the subdomain. Under this assumption, the 8×2 topology should be the fastest of the five tested topologies. The throughput ranking (Table 1, Figure 2) does not follow the indicators prediction. Instead, the two strip topologies (16×1 and 1×16) are the fastest overall and show the lowest communication share (8.3–roughly 9%), even though their χ values differ by more than a factor of three (1.22×10^{-3} versus 3.97×10^{-3}). Conversely, the three rectangular topologies cluster together at a distinctly higher communication share (10–11.1%) despite spanning a similarly wide range of χ (0.98×10^{-3} to 2.07×10^{-3}). In other words, topologies with the same neighbor count behave alike regardless of their χ value, while topologies with different neighbor counts behave differently even when their χ values coincide, as is the case for 16×1 and 4×4 (both 1.22×10^{-3})

Further the indicator fails to capture the asymmetry between the two strip orientations: 16×1 (decomposition along the long direction) is marginally faster than 1×16 (decomposition along the short direction), despite having a three times smaller χ value. These observations suggest that the number of MPI neighbors and the resulting communication pattern matter more than the total halo volume χ alone.

The higher communication cost of the rectangular topologies (Figure 3) is explained by two factors. Firstly, they require halo exchanges in both coordinate directions, doubling the number of MPI calls per iteration relative to the strips. Second, the subdomain corners must be packed and unpacked separately to maintain correct corner values after the two-pass exchange. These effects outweigh the lower boundary-to-volume ratio of the more balanced decompositions. The Nsight Systems traces in Appendix D confirm this behaviour: the halo phase of the 4×4 topology takes 822 μs per timestep, compared to 611 μs for the 16×1 strip.

The most likely explanation of the discrepancy between the expected and the measured behavior is that the communication stage in this benchmark is latency- rather than bandwidth-bound. The subdomains used here are large enough (thousands of cells per edge) that packed halo messages are not vanishingly small, but the dominant cost appears to be governed by the number of neighbor exchanges rather than by the total volume of halo data. Every additional MPI neighbor introduces an extra pack/unpack kernel launch, an extra point-to-point message, and an extra synchronization point, and these fixed, per-neighbor overheads outweigh the marginal bandwidth cost associated with a larger χ. It is therefore likely that for larger problems the findings described here do not hold unconditionally. This is also supported by the results of the overall performance experiment presented in Figure 4. It can be seen that the 1×16 topology outperforms most topologies up to a global problem size of roughly $2^{28.5}$ but then rapidly falls behind all other topologies. It is probable that the switch from latency- to bandwidth-bound conditions is observed at this threshold.

In the examined configuration, the modest rise in kernel time for the strip topologies is offset by the

decrease in communication time. The increase is most likely caused by diminished cache locality resulting from the more elongated subdomains. This aspect should be considered when transferring these findings to more complex problems.

5.2 Manual communication versus IGG

The overall performance comparison (Figure 4) shows comparable behaviour of both the manual decompositions as well as the IGG-based implementation. At small grid sizes, the manual implementation is able to outperform the IGG with all topologies. This suggests, that the IGG carries significant overhead at smaller sizes, which is not compensated by an increase in efficiency. Further, IGG favors topologies with a lower volume to edge ratio which as discussed above is not optimal for the chosen setup.

In contrast, the IGG implementation outperforms the manual implementation at problem sizes above 2^{27} cells. A likely explanation is that IGG uses nonblocking communication to overlap halo exchanges with kernel computation, whereas the manual implementation uses blocking *MPI.Sendrecv!*. This overlap gives IGG an advantage at intermediate grid sizes. At larger problem sizes both implementations converge to a throughput of approximately 44 GCell updates per second, or about 2.75 GCell/s per GPU. If the theoretical estimates in Appendix C are correct, this corresponds to around 26 percent of the roofline limit (166.7 GCell/s total), indicating that the solver operates well below the memory bandwidth ceiling and that the communication overhead becomes negligible relative to the kernel execution time.

The results confirm that the manual implementation is a valid alternative to the IGG implementation and enables a clearer understanding of the communication processes during the computation. At the same time, they indicate that for large problem sizes, performance can be further enhanced not only through topology choices but also through additional optimizations integrated into IGG.

5.3 Limitations

The set up was tested on a global grid with an aspect ratio of 1 by 4. This was chosen to allow for 5 unique topologies when parallelizing with 16 ranks. While these findings likely apply to square global domains, the complex trade-offs between limiting factors may yield a different optimal setup.

Further, in order to examine the communication time in the experiment, the communication had to be implemented blocking. A non-blocking implementation is possible, which would further reduce the cost of communication and importance of topology.

The bulk timing of the halo exchange and kernel calculations does not allow for detailed insight into the limiting factors. A stronger follow-up study could use MPI profiling or explicit timers around kernels, halo exchanges, and reductions to gain insight into further processes which could profit from optimization.

This experiment also focused on subdomains of equal size. A further optimization through the implementation of smaller subdomain sizes for ranks, which might limit the overall performance, could be investigated.

6 Conclusion

The primary research question asked how the MPI process topology influences the runtime and throughput of a multi-GPU shallow water solver. For a 4:1 global grid and 16 GPUs, the throughput differs by about two percent across the five possible topologies. The strip decompositions (16x1, 1x16) achieve the highest throughput, while the balanced 4x4 topology is the slowest. This ranking

is the opposite of what a simple boundary to volume ratio predicts: the number of MPI neighbors and the resulting communication pattern matter more than the total halo volume.

The secondary question asked whether the manually implemented MPI layer reaches performance comparable to the original IGG based solver. At large problem sizes both implementations converge to nearly identical throughput. The manual layer therefore introduces no significant overhead while retaining transparent control over decomposition and communication.

The communication share ranges from 8.3 to 11.1 percent of the iteration time. Kernel computation is constant across all topologies, confirming that the stencil work depends only on the global grid size. Future work could extend the sweep to square grids and other GPU counts, introduce nonblocking communication, and use GPU profiling to place the measured throughput on a roofline model. Nonuniform decompositions, where ranks own differently sized subdomains, would introduce a load imbalance dimension and allow a tradeoff between per rank work and communication cost.

Use of AI-Assisted Tools

The authors retain full ownership of the content and findings of this report. Language models were used as assistive tools for debugging Julia code, creating and styling plots, and improving the language of selected passages. All technical decisions, data interpretation, and the final text remain the responsibility of the authors.

A Reproducibility Information

A.1 Recommended software table

Table 2: Software and system information to record with the final benchmark.

Component	Version / configuration
System	CSCS Santis
GPU	NVIDIA GH200 (HBM3e, 141 GB)
Nodes / ranks	4 nodes / 16 MPI ranks
Ranks per node	4
GPUs per rank	1
Julia	1.12.6
ParallelStencil.jl	0.16.0
MPI.jl	0.20.26
MPI implementation	MPICH 5.0.1
CUDA.jl / CUDA toolkit	CUDA.jl 5.11.3 / CUDA 12.8.0
Floating-point type	Float64

B Domain Decomposition Visualisation

Figure 5 shows how the 4:1 global grid is divided into subdomains for each of the five process topologies used in the 16-rank benchmarks. The colouring indicates the rank ownership of each block.

C Theoretical Memory-Bandwidth Roofline

The theoretical cell-update throughput can be estimated from the GPU memory bandwidth as

$$P_{\max} = \frac{N_{\mathrm{GPU}} B_{\mathrm{GPU}}}{Q_{\mathrm{cell}}}, \tag{11}$$

where N_{GPU} is the number of GPUs, B_{GPU} is the theoretical memory bandwidth of one GPU in bytes per second, and Q_{cell} is the number of bytes transferred between GPU memory and the processor per complete cell update. The solver uses double-precision floating-point values (`Float64`), such that every array element occupies 8 bytes.

C.1 Optimistic memory-traffic estimate

An optimistic lower bound for Q_{cell} can be obtained by assuming that each required array element is transferred only once per kernel. This assumes ideal spatial reuse of stencil-neighbor values through registers and the GPU caches. The resulting traffic estimate is summarized in Table 3.

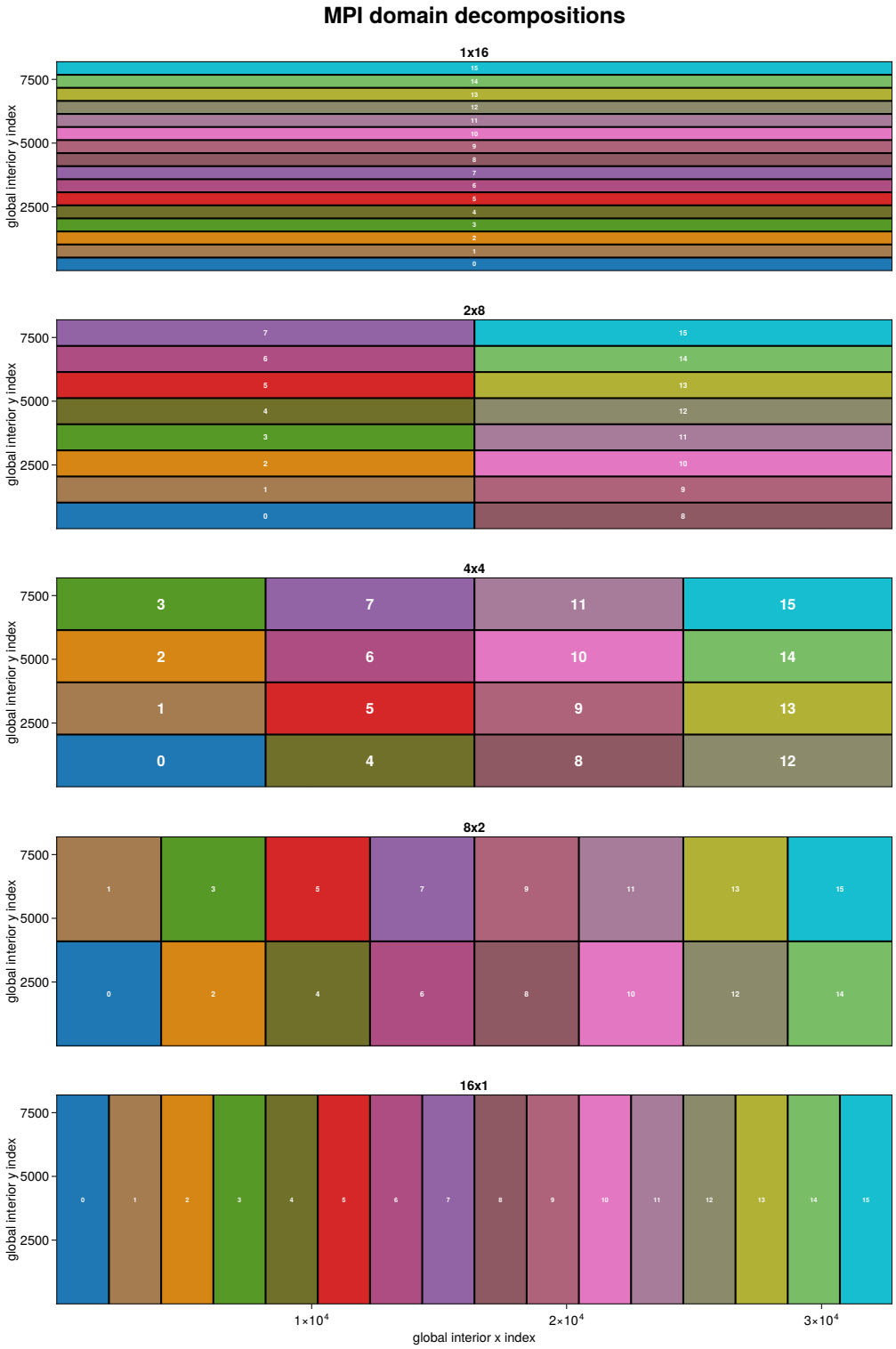


Figure 5: Domain decomposition of the global grid into the five subdomain layouts used for 16 ranks on a 4:1 global aspect ratio.

Table 3: Optimistic array traffic per cell and timestep.

Kernel	Reads	Writes	Values per cell
Maximum wave speed	h, z, hu, hv	s_x, s_y	6
Numerical fluxes	6	6	12
Draining timestep	3	1	4
Effective flux timesteps	3	2	5
State update	12	3	15
Two dry-cell corrections	6	0 normally	6
Total	$--$	$--$	48

The corresponding minimum memory traffic is therefore

$$Q_{\text{cell,min}} = 48 \times 8 = 384 \text{ bytes/cell}. \tag{12}$$

Substituting this value into Equation (11) gives the optimistic HBM-bandwidth roofline

$$P_{\text{max,optimistic}} = \frac{N_{\text{GPU}} B_{\text{GPU}}}{384 \text{ bytes/cell}}. \tag{13}$$

For example, for a GPU with a theoretical memory bandwidth of 4×10^{12} bytes/s, the maximum throughput per GPU is

$$P_{\text{max,GPU}} = \frac{4 \times 10^{12}}{384} \approx 10.4 \times 10^{9} \text{ cell updates/s}. \tag{14}$$

For 16 identical GPUs, the aggregate theoretical limit becomes

$$P_{\text{max,16}} = 16 \times 10.4 \approx 166.7 \text{ GCell updates/s}. \tag{15}$$

C.2 Conservative source-level estimate

A more conservative estimate can be obtained by counting the stencil-neighbor accesses visible in the kernel source code without assuming cache reuse. The approximate number of double-precision accesses is shown in Table 4.

Table 4: Conservative source-level traffic estimate per cell and timestep.

Kernel	`Float64` accesses per cell
Maximum wave speed	14
Numerical fluxes	24
Draining timestep	6
Effective flux timesteps	6
State update	27
Two dry-cell corrections	6
Total	83

This gives

$$Q_{\text{cell,conservative}} = 83 \times 8 = 664 \text{ bytes/cell}. \tag{16}$$

The expected memory traffic can consequently be approximated by the range

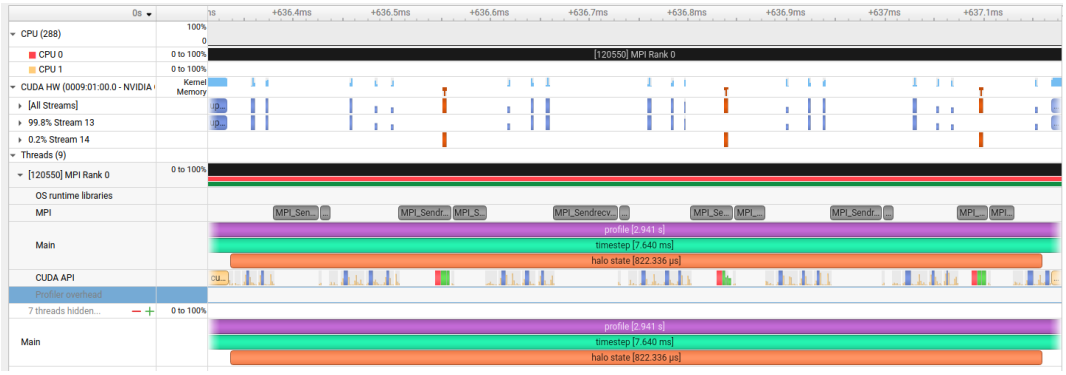
$$384 \lesssim Q_{\text{cell}} \lesssim 664 \qquad \text{bytes/cell}. \tag{17}$$

For a memory bandwidth of 4 TB/s, this range corresponds to approximately 6.0–10.4 GCell updates/s per GPU, or 96.4–166.7 GCell updates/s across 16 GPUs.

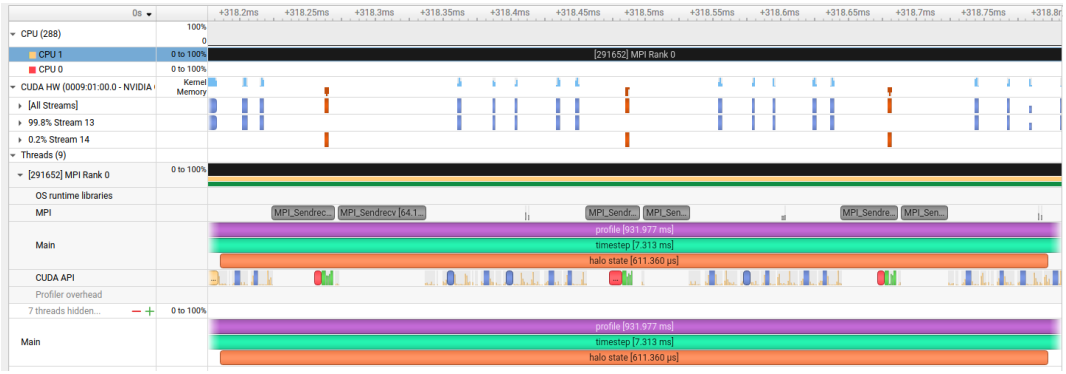
These estimates account only for bulk GPU-memory traffic. They exclude halo packing and unpacking, MPI communication, global reductions, physical boundary kernels, kernel-launch overhead, and additional writes triggered by dry cells. Consequently, the value based on 384 bytes/cell should be interpreted as an optimistic upper bound rather than an expected application throughput. A more accurate empirical roofline can be obtained by measuring the actual DRAM bytes transferred with a GPU profiler and dividing this value by the number of completed cell updates.

D Nsight Systems Profiling of Halo Exchange

The following Nsight Systems traces show the halo exchange phase of one timestep for two different topologies on rank 0.



(a) 4×4 topology: total halo time 822.34 μs.



(b) 16×1 topology: total halo time 611.36 μs.

Figure 6: Nsight Systems trace of one halo exchange phase for two topologies on rank 0. In the 4×4 decomposition, communication in both coordinate directions requires more MPI calls and exhibits non-contiguous memory layouts, leading to a longer total halo duration (822.34 μs). The 16×1 strip topology halves the number of messages and reduces the total time to 611.36 μs. However, individual calls in the 16×1 case show higher peak latencies (e.g. one `MPI_Sendrecv` reaching 64.1 μs) compared to the shorter, distributed calls in the 4×4 variant.

References

[1] R. Gassler and J. Käser, *ShallowWater4PDEonGPU: Parallel 2D shallow-water-equation solvers in Julia*, GitHub repository, 2026. Available: https://github.com/S1ntax3rror/

ShallowWater4PDEonGPU

[2] S. Omlin and L. Räss, "High-performance xPU stencil computations in Julia," *Proceedings of the JuliaCon Conferences*, vol. 6, no. 64, 2024. doi: 10.21105/jcon.00138.

[3] S. Omlin, *ParallelStencil.jl*, GitHub repository. Available: https://github.com/omlins/ParallelStencil.jl

[4] S. Omlin and L. Räss, "Distributed parallelization of xPU stencil computations in Julia," arXiv:2211.15716, 2022.

[5] ETH CSCS, *ImplicitGlobalGrid.jl*, GitHub repository. Available: https://github.com/eth-cscs/ImplicitGlobalGrid.jl

[6] MPI Forum, *MPI: A Message-Passing Interface Standard, Version 5.0*, 2025. Available: https://www.mpi-forum.org/docs/mpi-5.0/

[7] V. V. Rusanov, "Calculation of interaction of non-steady shock waves with obstacles," *Journal of Computational Mathematics and Mathematical Physics*, vol. 1, pp. 267–279, 1961.